

Tuples in Python

```
t = ( 1 , 2 , 'python' , tuple() , {42 , 'hi'} )
```

Diagram illustrating tuple indexing for `t`:

- `t[0]` points to `1`
- `t[1]` points to `2`
- `t[2]` points to `'python'`
- `t[3]` points to `tuple()`
- `t[4]` points to `{42, 'hi'}`

```
tuple = ('a', 1, 'e', 12.5, 'i', 'o', 9)
```

REVERSE INDEX

-7	-6	-5	-4	-3	-2	-1
a	1	e	12.5	i	o	9

FORWARD INDEX

0	1	2	3	4	5	6
---	---	---	---	---	---	---

Accessing Elements

```
tuple[0] = 'a'  
tuple[4] = 'i'  
tuple[-2] = 'o'  
tuple[-6] = 1
```

While accessing tuple elements, if you pass a negative index, Python adds the length of the tuple to the index to get element's forward index.

```
tuple[-5+6] = tuple[1] = 1
```



Python Tuples

CODE

```
tuple1 = tuple(("John", "Elena", "Albert"))  
print(tuple1)
```

OUTPUT

```
('John', 'Elena', 'Albert')
```

TUPLES

IMUTABLE

TUPLES → ()

musculo - spices = ("wordman", "chevy")

(spice 1, spice 2) = musculo - spices

print(f" Main musculo spices
{spice 1} {spice 2}")

⇓
wordman, chevy

ginger - ratio, wordman - ratio = 2, 1

print(f" Ratio is 0 {ginger - ratio}
and C: {wordman - ratio}")

⇓
0 : 2 C : 2

membership testing

```
print(f"Is ginger in masala  
spices? {'ginger' in masala_spices}")
```

⇓
FALSE

TUPLES în Python

1. Ce este un Tuple?

Tuple = colec ie de valori

Poate con ine mai multe tipuri de date

Este ordonat

Este IMUTABIL (nu poate fi modificat)

2. Cum se declar ?

Se folosesc paranteze rotunde () `t = (1, 2, 3)`

Sau cu tipuri diferite: `t = (10, "Ana", 3.14, True)`

3. Diferența față de list

List

`t = (1, 2, 3)`

Tuple

`t[0] = 10` # EROARE

[]

()

Mutabil

Imutabil

Poți modifica

NU poți modifica. De ce? Pentru că tuple este imutabil.

4. Accesare elemente (Index)

`t = (10, 20, 30)`

Exact ca la string și list .

`t[0]` 10

`t[1]` 20

5. Slicing

`t = (1, 2, 3, 4, 5)`

`t[-1]` 30

`t[1:4]` (2, 3, 4)

6. Tuple cu un singur element

Dacă scrii:

Trebuie virgulă !

`t = (5,)` # corect

`t = (5)`

Nu este tuple este int.

7. De ce folosim Tuple?

când datele NU trebuie modificate

pentru performanță (mai rapid decât list)

pentru date fixe (ex: coordonate)

Exemplu: `coordonate = (44.43, 26.10)`

8. Unpacking

Po i extrage valorile direct:

```
t = (1, 2)
```

```
a, b = t
```

```
print(a) # 1
```

```
print(b) # 2
```

Rezumat

1. Tuple = colec ie ordonat
2. Se declar cu ()
3. Este IMUTABIL
4. Suport index i slicing
5. Pentru un element (valoare,)

EXERCIIII TUPLE (cu rezolvare)

Rezolvare:

Exerci iul 1

```
t = (5, 10, 15, 20)
```

Ce afi eaz :

```
print(t[1])
```

t[1] 10

```
print(t[-1])
```

t[-1] 20

```
print(t[1:3])
```

t[1:3] (10, 15)

Exerci iul 2

```
t = (1, 2, 3)
```

Ce se întâmpl ?

```
t[0] = 100
```

Eroare TypeError

De ce?

Pentru c tuple este imutabil (nu poate fi modificat).

Exerci iul 3 – Unpacking

Rezultat:

```
coord = (44.43, 26.10)
```

```
lat = 44.43
```

```
lat, lon = coord
```

```
lon = 26.10
```

Se nume te unpacking

Tuple vs List

CARACTERISTICĂ	LIST	TUPLE
SINTAXĂ	[]	()
MUTABILĂ	DA	NU
PERFORMANȚĂ	LENTĂ	RAPIDĂ
CÂND FOLOSIM ?	DATE MODIFICABILE	DATE FIXE

Folosesc tuple când datele nu trebuie modificate, pentru siguranță și performanță mai

TUPLE

Se declară : (1, 2, 3)

Ordonat

Imutabil

Permite index

Permite slicing

Suport unpacking bun ."